# Practical Exam Practice Solutions

The HRMS scenario: A_1 to B_3

COS 110 | Swift Tuition | 2026 edition

Model solutions for the practice questions. Try each question first. Each solution is the one file a student would upload; it compiles with g++ -std=c++98 -Wall -Werror and runs clean under valgrind with the given files. Other correct answers exist.

## Contents


- **A_1 Exceptions**
  1. A1-01. Exceptions: Employee::setIDNum (length and gender) (100)
  2. A1-02. Exceptions: Employee::setIDNum (digits, month, day and gender) (100)
  3. A1-03. Exceptions: Employee::setIDNum (the age must match the birth year) (100)
  4. A1-04. Exceptions: Employee::extractIntFromString (100)
  5. A1-05. Exceptions: Team::operator[] throws for a bad index or an empty slot (100)
  6. A1-06. Exceptions: Department::operator[] with two error messages (100)
  7. A1-07. Exceptions: Team::addMember (NULL, already in the team, full) (100)
  8. A1-08. Exceptions: Team::assignIDs catches each error and continues (100)
  9. A1-09. Exceptions: Team::setMemberID catches two exception types (100)
  10. A1-10. Exceptions: Team::hire frees memory before it throws again (100)
- **A_2 Operator overloading**
  1. A2-01. Operator overloading: operator<< for Employee (100)
  2. A2-02. Operator overloading: operator<< for Team (skips empty slots) (100)
  3. A2-03. Operator overloading: operator<< for Department (virtual calls) (100)
  4. A2-04. Operator overloading: template operator<< for List<T> (100)
  5. A2-05. Operator overloading: Team::operator[] (const) returning const Employee& (100)
  6. A2-06. Operator overloading: Department::operator[] returning Team*& (100)
  7. A2-07. Operator overloading: Employee::operator== and operator!= (100)
  8. A2-08. Operator overloading: Department operator< and operator> by cost (100)
  9. A2-09. Operator overloading: Team::operator+= grows the array by one (100)
  10. A2-10. Operator overloading: Team::operator+ returns a merged team (100)
  11. A2-11. Operator overloading: Employee prefix and postfix operator++ (a birthday) (100)
  12. A2-12. Operator overloading: Team::operator() finds a surname (100)
- **A_3 Assignment operator**
  1. A3-01. Assignment operator: Employee::operator= (100)
  2. A3-02. Assignment operator: Team::operator= (100)
  3. A3-03. Assignment operator: Team::operator= with a compact copy (100)
  4. A3-04. Assignment operator: Department::operator= (100)
  5. A3-05. Assignment operator: Sales::operator= in a derived class (100)
  6. A3-06. Assignment operator: List<T>::operator= (100)
  7. A3-07. Copy constructor: Team::Team(const Team&) (100)
- **B_1 Recursion**
  1. B1-01. Recursion: Employee::extractIntFromString (100)
  2. B1-02. Recursion: Employee::isAllDigits (100)
  3. B1-03. Recursion: Team::countMembers (100)
  4. B1-04. Recursion: Department::totalEmployees (100)
  5. B1-05. Recursion: List<T>::countNodes (100)

6. B1-06. Recursion: List<T>::printReverse (100)
7. B1-07. Recursion: List<T>::contains (100)
8. B1-08. Recursion: Company::totalCost (100)
9. B1-09. Recursion: Employee::checksum (100)
10. B1-10. Recursion: Team::oldest (100)

- **B_2 Linked lists**
  1. B2-01. Linked lists: List<T>::append (100)
  2. B2-02. Linked lists: List<T>::insertAt (100)
  3. B2-03. Linked lists: List<T>::remove by value (100)
  4. B2-04. Linked lists: List<T>::removeAt (100)
  5. B2-05. Linked lists: List<T>::reverse (100)
  6. B2-06. Linked lists: List<T>::sort by relinking (100)
  7. B2-07. Linked lists: List<T>::indexOf (100)
  8. B2-08. Linked lists: List<T>::operator[] (100)
  9. B2-09. Linked lists: List<T>::clear (100)
  10. B2-10. Linked lists: Company::removeDepartment (ownership when removing from a list) (100)
  11. B2-11. Linked lists: List<T> copy constructor (100)
- **B_3 Template specialization**
  1. B3-01. Template specialization: printItem<Department*> (100)
  2. B3-02. Template specialization: largest<Employee> (100)
  3. B3-03. Template specialization: isEqual<Department*> (100)
  4. B3-04. Template specialization: List<Department*>::printAll (100)
  5. B3-05. Template specialization: List<Department*>::clear (100)
  6. B3-06. Template specialization: totalCost<Department*> (100)
  7. B3-07. Template specialization: addition<Employee, Employee> throws (100)

# A_1 Exceptions: solutions

## A1-01. Exceptions: `Employee::setIDNum` (length and gender) (100)

File: A_1.cpp

```
#include "Employee.h"

void Employee::setIDNum(string idNum) {
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
    // extractIntFromString throws "Not a digit" itself; that exception is not caught here.
    int genderDigits = extractIntFromString(idNum, 6, 10);
    char idGender = 'F';
    if (genderDigits >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
    this->idNum = idNum;
}
```

### Expected output of the given main.cpp

```
Thandi Mokoena, 29, F, ID: 9702140123081
Caught: ID number must have 13 digits
Caught: ID number does not match gender
Sipho Dlamini, 35, M, ID: none
Sipho Dlamini, 35, M, ID: 9105225800087
```

# A1-02. Exceptions: `Employee::setIDNum` (digits, month, day and gender) (100)

File: A_1.cpp

```
#include "Employee.h"

void Employee::setIDNum(string idNum) {
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
    for (int i = 0; i < (int)idNum.length(); i++) {
        if (idNum[i] < '0' || idNum[i] > '9') {
            throw Exception("ID number must contain only digits");
        }
    }
    int month = extractIntFromString(idNum, 2, 4);
    if (month < 1 || month > 12) {
        throw Exception("Invalid birth month");
    }
    int day = extractIntFromString(idNum, 4, 6);
    if (day < 1 || day > 31) {
        throw Exception("Invalid birth day");
    }
    char idGender = 'F';
    if (extractIntFromString(idNum, 6, 10) >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
    this->idNum = idNum;
}
```

## Expected output of the given main.cpp

```
Accepted: Thandi Mokoena, 29, F, ID: 9702140123081
Rejected 91052258000: ID number must have 13 digits
Rejected 91O5225800087: ID number must contain only digits
Rejected 9113225800087: Invalid birth month
Rejected 9105005800087: Invalid birth day
Rejected 9105220800087: ID number does not match gender
Accepted: Sipho Dlamini, 35, M, ID: 9105225800087
```

## A1-03. Exceptions: `Employee::setIDNum` (the age must match the birth year) (100)

File: A_1.cpp

```
#include "Employee.h"

void Employee::setIDNum(string idNum) {
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
    int yy = extractIntFromString(idNum, 0, 2);
    int birthYear = 1900 + yy;
    if (yy <= 26) {
        birthYear = 2000 + yy;
    }
    if (age != 2026 - birthYear) {
        throw Exception("ID number does not match age");
    }
    char idGender = 'F';
    if (extractIntFromString(idNum, 6, 10) >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
    this->idNum = idNum;
}
```

### Expected output of the given main.cpp

```
Accepted: Thandi Mokoena, 29, F, ID: 9702140123081
Accepted: Sipho Dlamini, 35, M, ID: 9105225800087
Rejected 8503305800089: ID number does not match gender
Accepted: Anna Botha, 41, F, ID: 8503300800089
Rejected 9001015800083: ID number does not match age
Accepted: Bongani Zulu, 30, M, ID: 9601015800083
```

# A1-04. Exceptions: `Employee::extractIntFromString` (100)

File: A_1.cpp

```
#include "Employee.h"

int Employee::extractIntFromString(string str, int start, int end) {
    if (start < 0 || start >= end || end > (int)str.length()) {
        throw Exception("Invalid range");
    }
    int result = 0;
    for (int i = start; i < end; i++) {
        if (str[i] < '0' || str[i] > '9') {
            throw Exception("Not a digit");
        }
        result = result * 10 + (str[i] - '0');
    }
    return result;
}
```

## Expected output of the given main.cpp

```
"9702140123081" [0, 2) = 97
"9702140123081" [2, 4) = 2
"9702140123081" [6, 10) = 123
"12a4" [0, 4): Not a digit
"1234" [2, 2): Invalid range
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: 9105225800087
```

## A1-05. Exceptions: `Team::operator[]` throws for a bad index or an empty slot (100)

File: A_1.cpp

```
#include "Team.h"

const Employee& Team::operator[](int index) {
    // Check the range first, so that members[index] is never read outside the array.
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    if (members[index] == NULL) {
        throw Exception("No employee at this index");
    }
    return *members[index];
}
```

### Expected output of the given main.cpp

```
[0] Thandi Mokoena, 29, F, ID: none
[1] Sipho Dlamini, 35, M, ID: none
[2] Caught: No employee at this index
[3] Caught: Index out of bounds
[-1] Caught: Index out of bounds
```

## A1-06. Exceptions: `Department::operator[]` with two error messages (100)

File: A_1.cpp

```
#include "Department.h"

Team*& Department::operator[](int index) {
    if (index < 0) {
        throw Exception("Negative index");
    }
    if (index >= size) {
        throw Exception("Index too large");
    }
    // An empty slot is not an error: the caller gets a reference to the NULL pointer.
    return teams[index];
}
```

### Expected output of the given main.cpp

```
Slot 1 is empty
Sales (cost R45000)
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team:
  Anna Botha, 41, F, ID: none
Caught: Negative index
Caught: Index too large
```

# A1-07. Exceptions: `Team::addMember` (NULL, already in the team, full) (100)

File: A_1.cpp

```
#include "Team.h"

void Team::addMember(Employee* employee) {
    if (employee == NULL) {
        throw Exception("Employee is NULL");
    }
    // Look at every slot first: the same pointer may sit after an empty slot.
    for (int i = 0; i < size; i++) {
        if (members[i] == employee) {
            throw Exception("Employee is already in the team");
        }
    }
    for (int i = 0; i < size; i++) {
        if (members[i] == NULL) {
            members[i] = employee;
            return;
        }
    }
    throw Exception("Team is full");
}
```

## Expected output of the given main.cpp

```
Caught: Employee is already in the team
Caught: Team is full
Caught: Employee is NULL
Members: 2
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
```

## A1-08. Exceptions: `Team::assignIDs` catches each error and continues (100)

File: A_1.cpp

```
#include "Team.h"

int Team::assignIDs(string* ids, int n) {
    if (ids == NULL || n <= 0) {
        return 0;
    }
    int count = 0;
    for (int i = 0; i < n; i++) {
        cout << "Slot " << i << ": ";
        if (i >= size) {
            cout << "no such slot" << endl;
        } else if (members[i] == NULL) {
            cout << "empty" << endl;
        } else {
            cout << members[i]->getFirstName() << " " << members[i]->getSurName() << " -> ";
            // The try is inside the loop, so one bad ID does not stop the others.
            try {
                members[i]->setIDNum(ids[i]);
                cout << ids[i] << endl;
                count++;
            } catch (Exception& e) {
                cout << "Error: " << e.what() << endl;
            }
        }
    }
    return count;
}
```

### Expected output of the given main.cpp

```
Slot 0: Thandi Mokoena -> 9702140123081
Slot 1: Sipho Dlamini -> Error: ID number does not match gender
Slot 2: empty
Slot 3: no such slot
IDs set: 1
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: none
```

### Hints given to students

## A1-08 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

### Hint 1

Each ID number needs its own chance to fail. Think about where a `try` block must start and end so that, after a `catch`, the function still goes on with the next `i`.

## Hint 2

Put the `try` and the `catch` **inside** the loop body. A `try` around the whole loop stops the loop at the first bad ID number.

## Hint 3

The two cases that are not errors (`i >= size` and an empty slot) do not need a `try`. Check them first. Add 1 to the count only after `setIDNum` returns without throwing.

# A1-09. Exceptions: `Team::setMemberID` catches two exception types (100)

File: A_1.cpp

```
#include "InvalidIDException.h"
#include "Team.h"

bool Team::setMemberID(int index, string idNum) {
    try {
        Employee* employee = getMember(index);
        employee->setIDNum(idNum);
        cout << "Member " << index << ": ID set to " << idNum << endl;
        return true;
    } catch (InvalidIDException& e) {
        // The most derived type must come first.
        cout << "Member " << index << ": bad ID [" << e.getBadID() << "]: " << e.what() << endl;
    } catch (Exception& e) {
        cout << "Member " << index << ": " << e.what() << endl;
    }
    return false;
}
```

### Expected output of the given main.cpp

```
Member 0: ID set to 9702140123081
Member 1: bad ID [91052258000]: ID number must have 13 digits
Member 1: bad ID [9105225800O87]: ID number must contain only digits
Member 1: ID number does not match gender
Member 2: No member at this index
Member 1: ID set to 9105225800087
IDs set: 2
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: 9105225800087
```

### Hints given to students

## A1-09 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

### Hint 1

An `InvalidIDException` object is also an `Exception` object. So a `catch (Exception& e)` block accepts both types.

### Hint 2

When an exception is thrown, C++ tries the `catch` blocks from top to bottom and uses the first one that fits. Ask yourself which of your two blocks would take an `InvalidIDException`.

### Hint 3

The `catch` block for the derived class must come before the `catch` block for its base class. In the opposite order, `-Wall -Werror` stops the build with "exception of type 'InvalidIDException' will be caught by earlier handler".

## A1-10. Exceptions: `Team::hire` frees memory before it throws again (100)

File: A_1.cpp

```
#include "Team.h"

void Team::hire(string firstName, string surName, int age, char gender, string idNum) {
    Employee* newEmployee = new Employee(firstName, surName, age, gender);
    try {
        newEmployee->setIDNum(idNum);
        addMember(newEmployee);
    } catch (Exception& e) {
        // The team did not take the employee, so it is freed here.
        delete newEmployee;
        throw e;  // "throw;" also works: it throws the same exception again
    }
}
```

### Expected output of the given main.cpp

```
Hired Thandi Mokoena
Caught: ID number does not match gender
Hired Sipho Dlamini
Caught: Team is full
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: 9105225800087
```

### Hints given to students

## A1-10 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

### Hint 1

Only steps 2 and 3 can throw. When one of them throws, which variable still points to the new employee, and who owns it at that moment?

### Hint 2

Put steps 2 and 3 in one `try` block. When the `catch (Exception& e)` block runs, the team does not hold the new employee, so `hire` is its only owner.

### Hint 3

A `catch` block can pass the exception on: `throw;` throws the caught exception again, and `throw e;` throws a copy of it. Free the memory before that line, because nothing after a `throw` runs.

# A_2 Operator overloading: solutions

## A2-01. Operator overloading: `operator<<` for `Employee` (100)

File: A_2.cpp

```
#include "Employee.h"

ostream& operator<<(ostream& out, const Employee& e) {
    // e is const and the getters are not const, so a friend reads the members directly.
    out << e.firstName << " " << e.surName << ", " << e.age << ", " << e.gender << ", ID: ";
    if (e.idNum == "") {
        out << "none";
    } else {
        out << e.idNum;
    }
    return out;
}
```

### Expected output of the given main.cpp

```
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: none
Thandi Mokoena, 29, F, ID: 9702140123081 | Sipho Dlamini, 35, M, ID: none
```

## A2-02. Operator overloading: `operator<<` for `Team` (skips empty slots) (100)

File: A_2.cpp

```
#include "Team.h"

ostream& operator<<(ostream& out, const Team& t) {
    // t is const, so getMemberCount() and operator[] (not const) cannot be called. Use the members.
    int count = 0;
    for (int i = 0; i < t.size; i++) {
        if (t.members[i] != NULL) {
            count++;
        }
    }
    out << "Team (" << count << "/" << t.size << "):" << endl;
    if (count == 0) {
        out << "  (no members)" << endl;
    }
    for (int i = 0; i < t.size; i++) {
        if (t.members[i] != NULL) {
            out << "  " << *t.members[i] << endl;
        }
    }
    return out;
}
```

### Expected output of the given main.cpp

```
Team (2/3):
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: none
Team (1/3):
  Sipho Dlamini, 35, M, ID: none
Team (0/2):
  (no members)
```

## A2-03. Operator overloading: `operator<<` for `Department` (virtual calls) (100)

File: A_2.cpp

```
#include "Department.h"

ostream& operator<<(ostream& out, Department& d) {
    // getName() and costToCompany() are virtual, so a Sales object prints as Sales.
    out << "Department: " << d.getName() << " (cost R" << d.costToCompany() << ")" << endl;
    bool anyTeam = false;
    for (int i = 0; i < d.size; i++) {
        if (d.teams[i] != NULL) {
            out << *d.teams[i];
            anyTeam = true;
        }
    }
    if (!anyTeam) {
        out << "  (no teams)" << endl;
    }
    return out;
}
```

### Expected output of the given main.cpp

```
Department: Development (cost R50000)
Team (2/2):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Department: HR (cost R12000)
Team (1/1):
  Anna Botha, 41, F, ID: none
```

# A2-04. Operator overloading: template operator<< for List<T> (100)

File: A_2.cpp

```
#ifndef A_2_CPP
#define A_2_CPP

#include "List.h"

template <class T>
ostream& operator<<(ostream& out, const List<T>& list) {
    if (list.headNode == NULL) {
        out << "Empty List";
        return out;
    }
    Node<T>* curr = list.headNode;
    // Stop at the last node, so that no "->" is printed after it.
    while (curr->next() != NULL) {
        out << curr->getData() << "->";
        curr = curr->next();
    }
    out << curr->getData();
    return out;
}

#endif
```

## Expected output of the given main.cpp

```
Empty List
1->2->3
Thandi->Sipho (size 2)
```

## A2-05. Operator overloading: `Team::operator[]` (const) returning `const Employee&` (100)

File: A_2.cpp

```
#include "Team.h"

const Employee& Team::operator[](int index) const {
    if (index < 0 || index >= size || members[index] == NULL) {
        throw Exception("Index out of bounds");
    }
    return *members[index];
}
```

### Expected output of the given main.cpp

```
First: Thandi Mokoena, 29, F, ID: none
Last: Anna Botha, 41, F, ID: none
Sipho Dlamini, 35, M, ID: none
Caught: Index out of bounds
```

## A2-06. Operator overloading: `Department::operator[]` returning `Team*&` (100)

File: A_2.cpp

```
#include "Department.h"

Team*& Department::operator[](int index) {
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    // Return the slot itself (not a copy), so the caller can change what it points to.
    return teams[index];
}
```

### Expected output of the given main.cpp

```
Development (cost R75000)
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team:
  Anna Botha, 41, F, ID: none
Development (cost R75000)
Team:
  Anna Botha, 41, F, ID: none
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Caught: Index out of bounds
```

## A2-07. Operator overloading: `Employee::operator==` and `operator!=` (100)

File: A_2.cpp

```
#include "Employee.h"

bool Employee::operator==(const Employee& other) const {
    // Only the ID numbers are compared. Two IDs that are not set ("") are equal too.
    return idNum == other.idNum;
}

bool Employee::operator!=(const Employee& other) const {
    return !(*this == other);
}
```

### Expected output of the given main.cpp

```
a, b (no IDs): == gives true, != gives false
a, b (same ID): == gives true, != gives false
a, c (different IDs): == gives false, != gives true
```

### Hints given to students

## A2-07 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

### Hint 1

`a != b` must always be the opposite of `a == b`. You do not need to compare the ID numbers a second time.

### Hint 2

Inside a member function, `*this` is the current object. So `*this == other` calls your own `operator==`.

### Hint 3

`operator!=` can return the opposite (`!`) of that call. Then the two operators can never disagree.

## A2-08. Operator overloading: `Department operator<` and `operator>` by cost (100)

File: A_2.cpp

```
#include "Department.h"

// costToCompany() is virtual, so each department uses its own cost rule.
bool Department::operator<(Department& other) {
    return costToCompany() < other.costToCompany();
}

bool Department::operator>(Department& other) {
    return costToCompany() > other.costToCompany();
}
```

### Expected output of the given main.cpp

```
Sales costs R30000
Development costs R25000
HR costs R0
Sales < Development: false
Sales > Development: true
Development < Sales: true
Development > Sales: false
HR < Development: true
HR > Development: false
```

## A2-09. Operator overloading: `Team::operator+=` grows the array by one (100)

File: A_2.cpp

```
#include "Team.h"

Team& Team::operator+=(Employee* employee) {
    if (employee == NULL) {
        return *this;
    }
    Employee** temp = new Employee*[size + 1];
    for (int i = 0; i < size; i++) {
        temp[i] = members[i];  // move the pointers; the employees stay where they are
    }
    temp[size] = new Employee(*employee);  // the team keeps its own copy
    delete[] members;                      // only the old array, not the employees
    members = temp;
    size++;
    return *this;
}
```

### Expected output of the given main.cpp

```
Team (2/2):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Main's employee: Sipho Dlamini, 35, M, ID: 9105225800087
Team (4/4):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
  Anna Botha, 41, F, ID: none
  Lebo Nkosi, 25, F, ID: none
```

## A2-10. Operator overloading: `Team::operator+` returns a merged team (100)

File: A_2.cpp

```
#include "Team.h"

Team Team::operator+(const Team& other) const {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL) {
            count++;
        }
    }
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            count++;
        }
    }

    Team result(count);
    int next = 0;
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL) {
            result.members[next] = new Employee(*members[i]);
            next++;
        }
    }
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            result.members[next] = new Employee(*other.members[i]);
            next++;
        }
    }
    return result;
}
```

### Expected output of the given main.cpp

```
Team (3/3):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
  Anna Botha, 41, F, ID: none
Team (2/3):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team (1/2):
  Anna Botha, 41, F, ID: none
```

## A2-11. Operator overloading: `Employee` prefix and postfix `operator++` (a birthday) (100)

File: A_2.cpp

```
#include "Employee.h"

// Prefix: ++e
Employee& Employee::operator++() {
    age++;
    return *this;
}

// Postfix: e++ (the int parameter only tells the two versions apart)
Employee Employee::operator++(int) {
    Employee old(*this);
    age++;
    return old;
}
```

### Expected output of the given main.cpp

```
Thandi Mokoena, 30, F, ID: none
Before: Thandi Mokoena, 30, F, ID: none
After: Thandi Mokoena, 31, F, ID: none
```

## A2-12. Operator overloading: `Team::operator()` finds a surname (100)

File: A_2.cpp

```
#include "Team.h"

int Team::operator()(string surName) const {
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL && members[i]->getSurName() == surName) {
            return i;
        }
    }
    return -1;
}
```

### Expected output of the given main.cpp

```
Dlamini: 1
Botha: 2
Nkosi: -1
```

# A_3 Assignment operator: solutions

## A3-01. Assignment operator: `Employee::operator=` (100)

File: A_3.cpp

```
#include "Employee.h"

Employee& Employee::operator=(const Employee& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }
    // 2. Free the old memory: Employee has no pointers, so there is nothing to delete.
    // 3. Copy every attribute from the other employee.
    firstName = other.firstName;
    surName = other.surName;
    age = other.age;
    gender = other.gender;
    idNum = other.idNum;
    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

### Expected output of the given main.cpp

```
Before:
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 34, M, ID: 9105225800087
After *thandi = *sipho:
Sipho Dlamini, 34, M, ID: 9105225800087
Sipho Dlamini, 34, M, ID: 9105225800087
```

## A3-02. Assignment operator: `Team::operator=` (100)

File: A_3.cpp

```
#include "Team.h"

Team& Team::operator=(const Team& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old memory: every employee, then the array.
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;

    // 3. Deep copy: a new array, and a new copy of every employee.
    size = other.size;
    members = new Employee*[size];
    for (int i = 0; i < size; i++) {
        if (other.members[i] == NULL) {
            members[i] = NULL;
        } else {
            members[i] = new Employee(*other.members[i]);
        }
    }

    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

### Expected output of the given main.cpp

```
Team a before:
Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Team b:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team a after *a = *b:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team b after removing member 0:
Team of size 3:
  0: empty
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team a has not changed:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
```

```
1: empty
2: Ravi Naidoo, 38, M, ID: none
```

## A3-03. Assignment operator: `Team::operator=` with a compact copy (100)

File: A_3.cpp

```
#include "Team.h"

Team& Team::operator=(const Team& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old memory: every employee, then the array.
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;

    // 3. The new size is the number of employees in the other team.
    int count = 0;
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            count++;
        }
    }
    size = count;
    members = new Employee*[size];

    // 4. Copy every employee into the next free slot, keeping the order.
    int next = 0;
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            members[next] = new Employee(*other.members[i]);
            next++;
        }
    }

    // 5. Return the current object so that a = b = c works.
    return *this;
}
```

### Expected output of the given main.cpp

```
Team a before:
Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Team b:
Team of size 4:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
  3: empty
Team a after *a = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
```

```
a: size 2, members 2
Team c after *d = *c = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
Team d after *d = *c = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
Team b after removing member 0:
Team of size 4:
  0: empty
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
  3: empty
Team a has not changed:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
```

## A3-04. Assignment operator: `Department::operator=` (100)

File: A_3.cpp

```
#include "Department.h"

Department& Department::operator=(const Department& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old memory: every team (each team deletes its employees), then the array.
    for (int i = 0; i < size; i++) {
        delete teams[i];
    }
    delete[] teams;

    // 3. Deep copy: a new array, and a new copy of every team.
    size = other.size;
    teams = new Team*[size];
    for (int i = 0; i < size; i++) {
        if (other.teams[i] == NULL) {
            teams[i] = NULL;
        } else {
            teams[i] = new Team(*other.teams[i]);
        }
    }

    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

### Expected output of the given main.cpp

```
Department a before:
Department of size 1:
Slot 0: Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Department b:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Department a after *a = *b:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Department b after the changes:
Department of size 3:
Slot 0: Team of size 2:
```

```
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
Slot 1: empty
Slot 2: empty
Department a has not changed:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
```

# A3-05. Assignment operator: `Sales::operator=` in a derived class (100)

File: A_3.cpp

```
#include "Sales.h"

Sales& Sales::operator=(const Sales& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }
    // 2. The base class copies its own part (the teams and the size).
    Department::operator=(other);
    // 3. Copy the member that Sales adds.
    target = other.target;
    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

## Expected output of the given main.cpp

```
Sales a before:
Sales of size 1:
Slot 0: Team of size 1:
  0: Thandi Mokoena, 29, F, ID: none
Target: 100, cost: R15000
Sales b:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
Sales a after *a = *b:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
Sales b after the changes:
Sales of size 2:
Slot 0: Team of size 2:
  0: empty
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 300, cost: R15000
Sales a has not changed:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
```

## Hints given to students

# A3-05 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

## Hint 1

Step 2 is the only new idea. A `Sales` object has a `Department` part, and that part has private attributes. Which function that is already written can copy them?

## Hint 2

Inside a member function of `Sales`, you can call a function of the base class by writing the class name and `::` in front of the function name. For example, `Department::getSize()` calls the `Department` version.

## Hint 3

A `Sales` object is a `Department`, so `other` (a `const Sales&`) can be passed where a `const Department&` is expected. The step-2 line is: `Department::operator=(other);`. Steps 1, 3 and 4 are the same as in any assignment operator.

## A3-06. Assignment operator: `List<T>::operator=` (100)

File: A_3.cpp

```
#ifndef A_3_CPP
#define A_3_CPP

#include "List.h"

template <class T>
List<T>& List<T>::operator=(const List<T>& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old nodes. Each node deletes its own data.
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr;
        curr = curr->next();
        delete temp;
    }
    headNode = NULL;
    size = 0;

    // 3. Copy the other list node by node, keeping the order.
    //    tail is the last node of this list, so each new node goes at the end.
    Node<T>* tail = NULL;
    Node<T>* otherCurr = other.headNode;
    while (otherCurr != NULL) {
        Node<T>* newNode = new Node<T>(new T(otherCurr->getData()));
        if (tail == NULL) {
            headNode = newNode;
        } else {
            tail->next() = newNode;
        }
        tail = newNode;
        size++;
        otherCurr = otherCurr->next();
    }

    // 4. Return the current object so that a = b = c works.
    return *this;
}

#endif
```

### Expected output of the given main.cpp

```
a: 9 (size 1)
b: 1->2->3 (size 3)
a after a = b: 1->2->3 (size 3)
b after b.prepend(0): 0->1->2->3 (size 4)
a has not changed: 1->2->3 (size 3)
```

# A3-07. Copy constructor: `Team::Team(const Team&)` (100)

File: A_3.cpp

```
#include "Team.h"

Team::Team(const Team& other) : members(NULL), size(0) {
    // A new object has no array yet. members is NULL and size is 0,
    // so operator= has nothing old to delete before it makes the deep copy.
    *this = other;
}
```

### Expected output of the given main.cpp

```
Team a:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
Team b, a copy of a:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
Team a after removing member 0:
Team of size 3:
  0: empty
  1: empty
  2: Anna Botha, 41, F, ID: none
Team b has not changed:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
```

### Hints given to students

## A3-07 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

### Hint 1

Think about the state of the new object when the copy constructor starts. What values do `members` and `size` hold before your code gives them values?

### Hint 2

If you reuse `operator=`, look at what the given `operator=` does first: it deletes every employee in `members` and then the array. In a brand-new object, those pointers were never set.

### Hint 3

Give `members` and `size` safe values before you reuse `operator=`: `NULL` and `0`. Deleting `NULL` does nothing, and a loop up to a size of `0` runs zero times. You can set them in the member initialiser list of the constructor.

# B_1 Recursion: solutions

## B1-01. Recursion: `Employee::extractIntFromString` (100)

File: B_1.cpp

```
#include "Employee.h"

int Employee::extractIntFromString(string str, int start, int end) {
    // Invalid range: return -1.
    if (start < 0 || start >= end || end > (int)str.length()) {
        return -1;
    }
    // The last character of the range must be a digit.
    if (str[end - 1] < '0' || str[end - 1] > '9') {
        return -1;
    }
    int lastDigit = str[end - 1] - '0';

    // Base case: the range has one character.
    if (end - start == 1) {
        return lastDigit;
    }

    // Recursive case: read the range without its last character.
    int front = extractIntFromString(str, start, end - 1);
    if (front == -1) {
        return -1;
    }
    return front * 10 + lastDigit;
}
```

### Expected output of the given main.cpp

```
Year: 97
Month: 2
Day: 14
Gender digits: 123
Bad range: -1
Not a digit: -1
Sipho Dlamini, 34, M, ID: 9105225800087
Caught: ID number must only contain digits
Caught: ID number must have 13 digits
Sipho Dlamini, 34, M, ID: 9105225800087
```

### Hints given to students

## B1-01 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

### Hint 1

Find the smallest valid range. An empty range (`start >= end`) is invalid, so the smallest valid range has one character, for example `(2, 3)`. For that range, return that one digit (or `-1` if it is not a digit). This is your base case, so the recursion must stop there and never call `(start, start)`.

## Hint 2

Now make the range one character shorter. The value of `"1234"` is the value of `"123"` times 10, plus 4. Which shorter range of `str` holds `"123"`?

## Hint 3

A `-1` from the shorter range means that range was not valid. The whole answer must then be `-1` too, so check the result of the recursive call before you multiply it by 10.

## B1-02. Recursion: `Employee::isAllDigits` (100)

File: B_1.cpp

```
#include "Employee.h"

bool Employee::isAllDigits(string str, int index) {
    // Invalid index: return false.
    if (index < 0 || index > (int)str.length()) {
        return false;
    }
    // Base case: no characters are left to check.
    if (index == (int)str.length()) {
        return true;
    }
    // This character is not a digit.
    if (str[index] < '0' || str[index] > '9') {
        return false;
    }
    // Recursive case: check the rest of the string.
    return isAllDigits(str, index + 1);
}
```

### Expected output of the given main.cpp

```
9702140123081 from 0: true
97021401230X1 from 0: false
abc123 from 3: true
abc123 from 0: false
Caught: ID number must only contain digits
Thandi Mokoena, 29, F, ID: 9702140123081
```

# B1-03. Recursion: `Team::countMembers` (100)

File: B_1.cpp

```
#include "Team.h"

int Team::countMembers(int index) {
    // Base case: an invalid index, or past the last slot. Nothing to count.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Count this slot if it holds an employee.
    int here = 0;
    if (members[index] != NULL) {
        here = 1;
    }
    // Recursive case: add the count of the remaining slots.
    return here + countMembers(index + 1);
}
```

## Expected output of the given main.cpp

```
Team of size 5:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
  3: Ravi Naidoo, 38, M, ID: none
  4: empty
Members from index 0: 3
Members from index 2: 2
Members from index 4: 0
After removing member 0, from index 0: 2
```

## B1-04. Recursion: `Department::totalEmployees` (100)

File: B_1.cpp

```
#include "Department.h"

int Department::totalEmployees(int index) {
    // Base case: an invalid index, or past the last slot. Nothing to add.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Employees in this slot (an empty slot adds 0).
    int here = 0;
    if (teams[index] != NULL) {
        here = teams[index]->getMemberCount();
    }
    // Recursive case: add the employees of the remaining slots.
    return here + totalEmployees(index + 1);
}
```

### Expected output of the given main.cpp

```
Department of size 4:
Slot 0: Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: Sipho Dlamini, 34, M, ID: none
  2: empty
Slot 1: empty
Slot 2: Team of size 2:
  0: Lebo Nkosi, 25, F, ID: none
  1: empty
Slot 3: empty
Employees from team 0: 3
Employees from team 2: 1
Employees from team 3: 0
After removing a member of team 0: 2
```

# B1-05. Recursion: `List<T>::countNodes` (100)

File: B_1.cpp

```
#ifndef B_1_CPP
#define B_1_CPP

#include "List.h"

template <class T>
int List<T>::countNodes(Node<T>* curr) {
    // Base case: past the last node, so there is nothing to count.
    if (curr == NULL) {
        return 0;
    }
    // Recursive case: this node, plus the nodes after it.
    return 1 + countNodes(curr->next());
}

#endif
```

## Expected output of the given main.cpp

```
10->20->30->40
Nodes from the head: 4
Nodes from the second node: 3
Nodes in an empty list: 0
```

## B1-06. Recursion: `List<T>::printReverse` (100)

File: B_1.cpp

```
#ifndef B_1_CPP
#define B_1_CPP

#include "List.h"

template <class T>
void List<T>::printReverse(Node<T>* curr) {
    // Base case: past the last node, so there is nothing to print.
    if (curr == NULL) {
        return;
    }
    // Recursive case: print the nodes after this one first, then this node.
    printReverse(curr->next());
    cout << curr->getData() << endl;
}

#endif
```

### Expected output of the given main.cpp

```
1->2->3
Reverse:
3
2
1
Reverse from the second node:
3
2
Reverse of an empty list:
(end)
```

## B1-07. Recursion: `List<T>::contains` (100)

File: B_1.cpp

```
#ifndef B_1_CPP
#define B_1_CPP

#include "List.h"

template <class T>
bool List<T>::contains(Node<T>* curr, T value) {
    // Base case: past the last node, so the value is not in the list.
    if (curr == NULL) {
        return false;
    }
    // Base case: this node holds the value.
    if (curr->getData() == value) {
        return true;
    }
    // Recursive case: search the nodes after this one.
    return contains(curr->next(), value);
}

#endif
```

### Expected output of the given main.cpp

```
5->10->15
Contains 10: true
Contains 7: false
Contains 15 (the last node): true
Contains 5, from the second node: false
Empty list contains 5: false
```

# B1-08. Recursion: `Company::totalCost` (100)

File: B_1.cpp

```
#include "Company.h"

float Company::totalCost(Node<Department*>* curr) {
    // Base case: past the last node, so there is nothing to add.
    if (curr == NULL) {
        return 0;
    }
    // Cost of the department in this node (a NULL department adds 0).
    float here = 0;
    if (curr->getData() != NULL) {
        here = curr->getData()->costToCompany();
    }
    // Recursive case: add the cost of the departments after this node.
    return here + totalCost(curr->next());
}
```

## Expected output of the given main.cpp

```
Company with 3 departments
HR of size 1:
Slot 0: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Sales of size 2:
Slot 0: Team of size 3:
  0: Anna Botha, 41, F, ID: none
  1: empty
  2: empty
Slot 1: Team of size 1:
  0: Ravi Naidoo, 38, M, ID: none
Development of size 1:
Slot 0: Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: Sipho Dlamini, 34, M, ID: none
Total cost: R92000
Cost from the second department: R80000
Cost from the third department: R50000
```

## B1-09. Recursion: `Employee::checksum` (100)

File: B_1.cpp

```
#include "Employee.h"

int Employee::checksum(string id, int index) {
    // Base case: an invalid index, or no digits left. Nothing to add.
    if (index < 0 || index >= (int)id.length()) {
        return 0;
    }
    int digit = id[index] - '0';

    // Double this digit if the number of characters after it is odd.
    // A doubled value of more than 9 becomes that value minus 9.
    if (((int)id.length() - 1 - index) % 2 == 1) {
        digit = digit * 2;
        if (digit > 9) {
            digit = digit - 9;
        }
    }

    // Recursive case: add the sum of the digits after this one.
    return digit + checksum(id, index + 1);
}
```

### Expected output of the given main.cpp

```
Luhn sum of 9702140123086: 50
Valid: yes
Luhn sum of 9702140123081: 45
Valid: no
Sum from index 6 of 9702140123086: 23
Caught: ID number has a wrong check digit
Thandi Mokoena, 29, F, ID: 9702140123086
```

## B1-10. Recursion: `Team::oldest` (100)

File: B_1.cpp

```
#include "Team.h"

int Team::oldest(int index) {
    // Base case: an invalid index, or past the last slot. No employees, so 0.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Recursive case: the oldest age in the slots after this one.
    int restOldest = oldest(index + 1);

    // Compare this slot (if it holds an employee) with the rest.
    if (members[index] != NULL && members[index]->getAge() > restOldest) {
        return members[index]->getAge();
    }
    return restOldest;
}
```

### Expected output of the given main.cpp

```
Team of size 5:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
  3: Ravi Naidoo, 38, M, ID: none
  4: empty
Oldest from index 0: 41
Oldest from index 3: 38
Oldest from index 4: 0
After removing member 2, from index 0: 38
```

# B_2 Linked lists: solutions

## B2-01. Linked lists: `List<T>::append` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Adds value at the end of the list.
template <class T>
void List<T>::append(T value) {
    Node<T>* newNode = new Node<T>(new T(value));
    // Case 1: the list is empty, so the new node becomes the head.
    if (headNode == NULL) {
        headNode = newNode;
        size++;
        return;
    }
    // Case 2: walk to the last node (the node whose next is NULL).
    Node<T>* curr = headNode;
    while (curr->next() != NULL) {
        curr = curr->next();
    }
    curr->next() = newNode;
    size++;
}

#endif
```

### Expected output of the given main.cpp

```
Empty List (size 0)
10 (size 1)
10->20->30 (size 3)
5->10->20->30->40 (size 5)
```

# B2-02. Linked lists: `List<T>::insertAt` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Inserts value so that it ends up at index pos (the first node is index 0).
// pos <= 0 adds at the front. pos >= size adds at the back.
template <class T>
void List<T>::insertAt(T value, int pos) {
    Node<T>* newNode = new Node<T>(new T(value));
    // Front: an empty list, or a position of 0 or less.
    if (headNode == NULL || pos <= 0) {
        newNode->next() = headNode;
        headNode = newNode;
        size++;
        return;
    }
    // Move prev to the node just before index pos.
    // If the list runs out first, prev stops at the last node (append).
    Node<T>* prev = headNode;
    int count = 1;
    while (prev->next() != NULL && count < pos) {
        prev = prev->next();
        count++;
    }
    newNode->next() = prev->next();
    prev->next() = newNode;
    size++;
}

#endif
```

## Expected output of the given main.cpp

```
20 (size 1)
20->40 (size 2)
20->30->40 (size 3)
10->20->30->40 (size 4)
10->20->30->35->40 (size 5)
10->20->30->35->40->50 (size 6)
```

# B2-03. Linked lists: `List<T>::remove` by value (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Removes the first node that holds value. Returns true if a node was removed.
template <class T>
bool List<T>::remove(T value) {
    // Case 1: empty list, nothing to remove.
    if (headNode == NULL) {
        return false;
    }
    // Case 2: the head holds the value.
    if (headNode->getData() == value) {
        Node<T>* temp = headNode;
        headNode = headNode->next();
        delete temp;    // ~Node() deletes the data
        size--;
        return true;
    }
    // Case 3: look further, keeping prev one node behind curr.
    // The loop checks curr != NULL first, so a one-node list is safe.
    Node<T>* prev = headNode;
    Node<T>* curr = headNode->next();
    while (curr != NULL) {
        if (curr->getData() == value) {
            prev->next() = curr->next();
            delete curr;
            size--;
            return true;
        }
        prev = curr;
        curr = curr->next();
    }
    // Case 4: not found.
    return false;
}

#endif
```

## Expected output of the given main.cpp

```
1->2->3->4->5 (size 5)
remove(3): true -> 1->2->4->5 (size 4)
remove(1): true -> 2->4->5 (size 3)
remove(5): true -> 2->4 (size 2)
remove(9): false -> 2->4 (size 2)
```

# B2-04. Linked lists: `List<T>::removeAt` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Removes the node at index pos (the first node is index 0).
// Returns false and changes nothing when pos is out of range.
template <class T>
bool List<T>::removeAt(int pos) {
    // Out of range (this also covers an empty list).
    if (pos < 0 || pos >= size) {
        return false;
    }
    // Remove the head.
    if (pos == 0) {
        Node<T>* temp = headNode;
        headNode = headNode->next();
        delete temp;
        size--;
        return true;
    }
    // Move prev to the node just before index pos.
    Node<T>* prev = headNode;
    for (int i = 0; i < pos - 1; i++) {
        prev = prev->next();
    }
    Node<T>* curr = prev->next();
    prev->next() = curr->next();
    delete curr;
    size--;
    return true;
}

#endif
```

## Expected output of the given main.cpp

```
10->20->30->40->50 (size 5)
removeAt(2): true -> 10->20->40->50 (size 4)
removeAt(0): true -> 20->40->50 (size 3)
removeAt(2): true -> 20->40 (size 2)
removeAt(5): false -> 20->40 (size 2)
removeAt(-1): false -> 20->40 (size 2)
```

## B2-05. Linked lists: `List<T>::reverse` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Reverses the order of the nodes by turning every link around.
// No node is created or deleted.
template <class T>
void List<T>::reverse() {
    Node<T>* prev = NULL;
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr->next();   // remember the rest of the list
        curr->next() = prev;            // turn this link around
        prev = curr;
        curr = temp;
    }
    headNode = prev;                    // the old last node is the new head
}

#endif
```

### Expected output of the given main.cpp

```
1->2->3->4->5 (size 5)
5->4->3->2->1 (size 5)
1->2->3->4->5 (size 5)
```

# B2-06. Linked lists: `List<T>::sort` by relinking (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Sorts the list in ascending order by relinking the nodes (insertion sort).
// The data cannot be changed, so whole nodes are moved instead.
template <class T>
void List<T>::sort() {
    Node<T>* sorted = NULL;           // head of the sorted part
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr->next(); // remember the rest of the unsorted part
        // Case 1: curr goes in front of the sorted part.
        if (sorted == NULL || curr->getData() < sorted->getData()) {
            curr->next() = sorted;
            sorted = curr;
        } else {
            // Case 2: find the last sorted node that is <= curr, and link curr after it.
            Node<T>* prev = sorted;
            while (prev->next() != NULL && prev->next()->getData() <= curr->getData()) {
                prev = prev->next();
            }
            curr->next() = prev->next();
            prev->next() = curr;
        }
        curr = temp;
    }
    headNode = sorted;
}

#endif
```

## Expected output of the given main.cpp

```
0->5->3->6->-1 (size 5)
-1->0->3->5->6 (size 5)
```

# B2-07. Linked lists: `List<T>::indexOf` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Returns the index of the first node that holds value, or -1 if there is none.
template <class T>
int List<T>::indexOf(T value) {
    Node<T>* curr = headNode;
    int index = 0;
    while (curr != NULL) {
        if (curr->getData() == value) {
            return index;   // early exit: the first match wins
        }
        curr = curr->next();
        index++;
    }
    return -1;              // empty list, or not found
}

#endif
```

## Expected output of the given main.cpp

```
Sales->Marketing->HR->Development
indexOf(Sales): 0
indexOf(HR): 2
indexOf(Development): 3
indexOf(Finance): -1
```

# B2-08. Linked lists: `List<T>::operator[]` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Returns the data in the node at position index (the head is position 0).
// Throws an Exception when index is out of range.
template <class T>
const T& List<T>::operator[](int index) {
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    Node<T>* curr = headNode;
    for (int i = 0; i < index; i++) {
        curr = curr->next();
    }
    return curr->getData();
}

#endif
```

## Expected output of the given main.cpp

```
10->20->30
list[0]: 10
list[2]: 30
list[1]: 20
list[3]: Caught: Index out of bounds
list[-1]: Caught: Index out of bounds
```

# B2-09. Linked lists: `List<T>::clear` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Deletes every node. The list is empty afterwards and can be used again.
template <class T>
void List<T>::clear() {
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr;      // never delete the loop variable itself
        curr = curr->next();
        delete temp;               // ~Node() deletes the data
    }
    headNode = NULL;
    size = 0;
}

#endif
```

## Expected output of the given main.cpp

```
1->2->3->4 (size 4)
Empty List (size 0)
8->7 (size 2)
```

## B2-10. Linked lists: `Company::removeDepartment` (ownership when removing from a list) (100)

File: B_2.cpp

```
#include "Company.h"

// Removes the first department (in list order) whose name is name.
// The company owns its departments, so the Department object is deleted too.
bool Company::removeDepartment(string name) {
    Node<Department*>* curr = departments.head();
    while (curr != NULL) {
        Department* department = curr->getData();
        if (department->getName() == name) {
            // Unlink and delete the node first (curr is not used after this),
            // then delete the Department that the pointer points to.
            departments.remove(department);
            delete department;
            return true;
        }
        curr = curr->next();
    }
    return false;
}
```

### Expected output of the given main.cpp

```
Company with 3 departments
Development (cost R25000)
Team:
  Lebo Nkosi, 25, F, ID: none
HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R64000
removeDepartment(HR): true
removeDepartment(Finance): false
Company with 2 departments
Development (cost R25000)
Team:
  Lebo Nkosi, 25, F, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R40000
```

# B2-11. Linked lists: `List<T>` copy constructor (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Copy constructor: builds a deep copy of other, in the same order.
template <class T>
List<T>::List(const List<T>& other) : headNode(NULL), size(0) {
    Node<T>* tail = NULL;              // last node of the new list
    Node<T>* curr = other.headNode;
    while (curr != NULL) {
        // A new node with a new copy of the data.
        Node<T>* newNode = new Node<T>(new T(curr->getData()));
        if (tail == NULL) {
            headNode = newNode;        // first node of the copy
        } else {
            tail->next() = newNode;    // link after the current last node
        }
        tail = newNode;
        size++;
        curr = curr->next();
    }
}

#endif
```

## Expected output of the given main.cpp

```
original: 1->2->3 (size 3)
copy:     1->2->3 (size 3)
original: 0->1->2->3 (size 4)
copy:     9->1->2->3 (size 4)
```

# B_3 Template specialization: solutions

## B3-01. Template specialization: `printItem<Department*>` (100)

File: B_3.cpp

```
#include "Templates.h"

// For a Department pointer, print the Department it points to, not the address.
template <>
void printItem<Department*>(Department* const& item) {
    if (item == NULL) {
        cout << "No department" << endl;
        return;
    }
    cout << *item;    // Department's operator<< already ends its lines
}
```

### Expected output of the given main.cpp

```
42
3.5
Yyra HRMS
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
No department
```

## B3-02. Template specialization: `largest<Employee>` (100)

File: B_3.cpp

```
#include "Templates.h"

// For employees, the "largest" one is the older one. Same age: return b,
// like the general template does when the values are equal.
template <>
Employee largest<Employee>(Employee a, Employee b) {
    if (a.getAge() > b.getAge()) {
        return a;
    }
    return b;
}
```

### Expected output of the given main.cpp

```
7
2.5
pear
Sipho Dlamini, 34, M, ID: none
Sipho Dlamini, 34, M, ID: none
```

## B3-03. Template specialization: `isEqual<Department*>` (100)

File: B_3.cpp

```
#include "Templates.h"

// Two departments are equal when they cost the company the same amount.
template <>
bool isEqual<Department*>(Department* a, Department* b) {
    if (a == NULL && b == NULL) {
        return true;
    }
    if (a == NULL || b == NULL) {
        return false;
    }
    return a->costToCompany() == b->costToCompany();
}
```

### Expected output of the given main.cpp

```
5 and 5: equal
5 and 6: not equal
HR and HR: equal
Marketing R36000, HR R36000, Sales R15000
Marketing and HR: equal
Sales and HR: not equal
```

## B3-04. Template specialization: `List<Department*>::printAll` (100)

File: B_3.cpp

```
#include "Department.h"
#include "List.h"

// For a list of Department pointers, print each Department, not its address.
template <>
void List<Department*>::printAll() {
    if (headNode == NULL) {
        cout << "Empty List" << endl;
        return;
    }
    Node<Department*>* curr = headNode;
    int index = 0;
    while (curr != NULL) {
        Department* department = curr->getData();
        if (department == NULL) {
            cout << index << ": No department" << endl;
        } else {
            cout << index << ": " << *department;   // operator<< ends its own lines
        }
        curr = curr->next();
        index++;
    }
}
```

### Expected output of the given main.cpp

```
0: 10
1: 20
2: 30
0: HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
1: Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
```

## B3-05. Template specialization: `List<Department*>::clear` (100)

File: B_3.cpp

```
#include "Department.h"
#include "List.h"

// For a list of Department pointers, the list owns the Departments:
// delete each Department, then its node.
template <>
void List<Department*>::clear() {
    Node<Department*>* curr = headNode;
    while (curr != NULL) {
        Node<Department*>* temp = curr;
        curr = curr->next();
        delete temp->getData();   // the Department (deleting NULL does nothing)
        delete temp;              // the node, which deletes its copy of the pointer
    }
    headNode = NULL;
    size = 0;
}
```

### Expected output of the given main.cpp

```
1->2->3
Empty List (size 0)
Company with 2 departments
HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R39000
Spare departments: 2
Spare departments after clear: 0
```

## B3-06. Template specialization: `totalCost<Department*>` (100)

File: B_3.cpp

```
#include "Templates.h"

// The total monthly cost of every department in the list. NULL entries are skipped.
template <>
float totalCost<Department*>(List<Department*>& list) {
    float total = 0;
    Node<Department*>* curr = list.head();
    while (curr != NULL) {
        Department* department = curr->getData();
        if (department != NULL) {
            total += department->costToCompany();   // virtual: the subclass rule is used
        }
        curr = curr->next();
    }
    return total;
}
```

### Expected output of the given main.cpp

```
totalCost(numbers): 0
Company cost: R76000
totalCost(empty): 0
```

## B3-07. Template specialization: `addition<Employee, Employee>` throws (100)

File: B_3.cpp

```
#include "Templates.h"

// Adding two employees makes no sense, so this specialization throws.
template <>
Employee addition<Employee, Employee>(Employee v1, Employee v2) {
    throw Exception("Cannot add two employees");
}
```

### Expected output of the given main.cpp

```
7
5.5
Human Resources
Caught: Cannot add two employees
Program continues
```